

Instalación del entorno por sistema operativo

Curso MLOps/DataOps · ANBAN

José Picón

2026

Antes de empezar: qué vas a instalar y por qué

Este capítulo te lleva desde “ordenador recién encendido” hasta “todo el laboratorio del curso levantado”. Cada paso va explicado: qué hace, qué tienes que ver en pantalla, qué hacer si algo no sale como esperas.

Cuando termines este capítulo, tendrás:

- **Docker Desktop** (Windows y macOS) o **Docker Engine** (Linux) instalado y funcionando. Es lo que permite ejecutar contenedores, que son cajas autocontenidas con software listo para usar.
- **Una terminal** donde teclear los comandos del curso. En Linux y macOS viene de serie. En Windows hay dos opciones: PowerShell (nativo) o WSL (un Linux dentro de Windows).
- **Cinco contenedores arrancados** que forman el laboratorio: Jupyter, MLflow, MinIO, PostgreSQL y un inicializador de buckets.

No necesitas instalar Python, ni Git, ni librerías sueltas en el sistema. Todo eso va dentro de los contenedores. Eso es la gracia de Docker: una vez instalado, todo lo demás “ya viene”.

Tiempo estimado de instalación completa: 30 minutos la primera vez. La segunda, 2 minutos (solo levantar lo que ya descargaste).

A • Windows 10 / Windows 11

En Windows tienes dos caminos. Lee los dos antes de decidir.

Opción	Cuándo elegirla	Ventajas	Inconvenientes
WSL (Ubuntu dentro de Windows)	Recomendado. Si no tienes restricción de tu empresa.	Es Linux real, rendimiento óptimo de Docker. Los comandos del curso funcionan tal cual.	Hay que instalar WSL (10 minutos) la primera vez.
PowerShell nativo	Si no puedes/no quieres instalar WSL	No instalas nada extra de sistema.	Algunos comandos del curso requieren adaptación. Docker más lento con volúmenes.

Si dudas, escoge **WSL**. El resto de esta sección lo explica.

A.1 Instalar WSL (Windows Subsystem for Linux)

WSL es Linux ejecutándose dentro de Windows, sin emulación. Tu ordenador no se reinicia en otro sistema; conviven los dos.

Paso 1.1 — Comprobar que tienes Windows 10 (build 19041+) o Windows 11

Pulsa la tecla Windows + R. En la ventana “Ejecutar”, escribe:

```
winver
```

Pulsa Enter. Se abre una ventana con tu versión. Lee la línea “Versión 19041” (o superior) o “Versión 22H2”, etc. Si la versión es inferior a 19041, primero actualiza Windows (Inicio → Configuración → Windows Update → Buscar actualizaciones).

Paso 1.2 — Abrir PowerShell como administrador

Pulsa la tecla Windows. Escribe:

```
powershell
```

En los resultados, sobre “Windows PowerShell”, **botón derecho** → **Ejecutar como administrador**. Windows te pedirá permiso, di que sí. Aparece una ventana azul oscuro con texto blanco.

Paso 1.3 — Lanzar la instalación de WSL

En esa ventana de PowerShell teclea exactamente:

```
wsl --install -d Ubuntu
```

Pulsa Enter. Verás mensajes parecidos a:

```
Installing: Virtual Machine Platform
Virtual Machine Platform has been installed.
Installing: Windows Subsystem for Linux
...
Installing: Ubuntu
Ubuntu has been installed.
The requested operation is successful.
Changes will not be effective until the system is rebooted.
```

El proceso tarda entre 2 y 5 minutos según tu red. Espera a que termine.

Paso 1.4 — Reiniciar el ordenador

Cuando termine, **reinicia el ordenador completamente** (no es un cierre de sesión, es un reinicio normal). Sin reiniciar no funcionará.

Paso 1.5 — Primer arranque de Ubuntu

Tras reiniciar, Windows abre **automáticamente** una ventana negra que dice:

```
Installing, this may take a few minutes...
```

Espera. Cuando termina, te pregunta:

```
Enter new UNIX username:
```

Escribe un nombre de usuario en minúsculas, sin acentos, sin espacios. Por ejemplo: `jose` o `alumno`. Pulsa Enter.

A continuación pide:

```
New password:
```

Escribe una contraseña. **No verás los caracteres que tecleas**: ni asteriscos, ni nada. Es así en Linux por seguridad. Teclea a ciegas y pulsa Enter. Te la pedirá una segunda vez para confirmar.

Cuando termina ves el prompt verde:

```
jose@MIPC:~$
```

Ya estás dentro de Ubuntu. Eso significa: estás en un Linux dentro de Windows.

Si te aparece el error 0x80370102

Te lo dice así:

```
WslRegisterDistribution failed with error: 0x80370102
Please enable the Virtual Machine Platform Windows feature
and ensure virtualization is enabled in the BIOS.
```

Quiere decir que **falta activar la virtualización**. Pasos:

1. **Comprueba la virtualización en el sistema.** Pulsa Ctrl + Shift + Esc para abrir el Administrador de tareas. Ve a la pestaña **Rendimiento** → **CPU**. Mira abajo a la derecha la línea **Virtualización**:
 - Si pone **Habilitado**: el problema es solo de Windows. Sigue al siguiente punto.
 - Si pone **Deshabilitado**: hay que activarla en la BIOS. Salta al punto “Activar virtualización en la BIOS” más abajo.
2. **Activa las características de Windows.** Abre PowerShell como administrador y ejecuta los dos comandos:

```
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-
Linux /all /norestart
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /
all /norestart
```

Cuando acaban (30-60 segundos), **reinicia el ordenador completo**.

3. **Vuelve a lanzar WSL:** tras reiniciar, abre PowerShell normal (no hace falta admin) y ejecuta:

```
wsl --install -d Ubuntu
```

Activar virtualización en la BIOS

Si el Administrador de tareas decía “Deshabilitado”:

1. **Reinicia el ordenador** y aporrea la tecla de BIOS en cuanto veas el logo del fabricante. Las teclas habituales:

Marca	Tecla
HP	F10 o Esc
Dell	F2 o F12
Lenovo	F1, F2 o Enter luego F1
ASUS	F2 o Del
Acer	F2 o Del
MSI	Del
Sobremesa genérico	Del o F2

2. **Busca la opción de virtualización.** Está en una de estas secciones según tu BIOS:

- “Advanced” → “CPU Configuration”
- “Configuration” → “Virtualization”
- “Security” → “Virtualization”
- “System Configuration”

Y el nombre exacto cambia:

- Procesador Intel: **Intel Virtualization Technology** o **VT-x**.
- Procesador AMD: **SVM Mode**, **AMD-V** o **SVM**.

Cámbialo a **Enabled**.

3. **Guarda y sal** (suele ser F10, luego confirmar). El equipo reinicia. Vuelve al Paso 1.5.

A.2 Instalar Docker Desktop

Docker Desktop es la aplicación que gestiona los contenedores.

Paso 2.1 — Descargar Docker Desktop

Abre tu navegador y entra en:

<https://www.docker.com/products/docker-desktop/>

Pulsa el botón **Download for Windows - AMD64** (o **ARM64** si tu PC es ARM, raro). Se descarga un fichero `Docker Desktop Installer.exe` de unos 600 MB.

Paso 2.2 — Ejecutar el instalador

Doble click en el `.exe` descargado. Aparece una ventana de configuración:

- **Use WSL 2 instead of Hyper-V:** déjalo **marcado**. Es lo que quieres.

- **Add shortcut to desktop:** como prefieras.

Pulsa **OK**. La instalación tarda entre 3 y 5 minutos.

Paso 2.3 — Cerrar sesión de Windows

Cuando termina, te pide cerrar sesión. No es un reinicio: solo cerrar sesión y volver a iniciarla con tu usuario. Hazlo.

Paso 2.4 — Abrir Docker Desktop por primera vez

Tras volver a iniciar sesión, abre Docker Desktop desde el menú Inicio. La primera vez:

- Te muestra un acuerdo de licencia: acéptalo (es gratuito para uso personal y educativo).
- Empieza a arrancar el motor. Verás el icono de la **ballena de Docker** en la bandeja del sistema (abajo a la derecha, junto al reloj). Mientras está animada (las olas suben y bajan), Docker está arrancando.

Espera hasta que la ballena se **quede fija** (sin animación). En ese momento, Docker está listo. Suele tardar 1-2 minutos.

Paso 2.5 — Conectar Docker con WSL (importante)

Si has elegido la ruta WSL, Docker Desktop tiene que “ver” tu Ubuntu. Esto se olvida siempre y luego falla todo.

1. Abre Docker Desktop.
2. Engranaje arriba a la derecha (**Settings**).
3. En el menú lateral: **Resources** → **WSL integration**.
4. Marca la casilla “**Enable integration with my default WSL distro**”.
5. En la lista de abajo, encuentra **Ubuntu** y activa su interruptor.
6. Pulsa **Apply & Restart**.

Espera 30 segundos a que Docker reinicie.

Paso 2.6 — Verificar Docker desde Ubuntu

Abre **Ubuntu** desde el menú Inicio (no PowerShell). En el prompt verde, teclea:

```
docker --version
```

Debes ver algo así:

```
Docker version 25.0.3, build 4debf41
```

Después:

```
docker run --rm hello-world
```

Docker descarga una imagen pequeña de prueba y la ejecuta. Al final verás:

```
Hello from Docker!  
This message shows that your installation appears to be working correctly.
```

Si llegas aquí, Docker funciona perfectamente.

A.3 Si no quieres usar WSL: PowerShell nativo

Si has decidido **no** usar WSL, instalas Docker Desktop igual (Paso A.2) pero **NO** marcas la **integración WSL**. Trabajarás siempre desde PowerShell.

Para arrancar PowerShell en tu carpeta del curso, escribe en el menú Inicio “PowerShell” y abre la app **Windows PowerShell** (no la versión “ISE”). Te aparece la ventana azul con prompt:

```
PS C:\Users\TuNombre>
```

Para verificar:

```
docker --version  
docker run --rm hello-world
```

Mismas salidas que antes.

B · macOS (Intel y Apple Silicon)

En macOS solo necesitas Docker Desktop. La terminal viene de serie.

B.1 Identifica tu chip

Manzana arriba izquierda → **Acerca de este Mac**. Mira la línea “Chip” o “Procesador”:

- Si dice **Apple M1, M2, M3 o M4**: tu chip es ARM64 (Apple Silicon).
- Si dice **Intel Core i5/i7/i9** o similar: tu chip es Intel x86_64.

Esto importa para descargar la versión correcta de Docker.

B.2 Instalar Docker Desktop

Opción rápida con Homebrew (recomendado si ya lo tienes)

Si ya usas Homebrew (gestor de paquetes de macOS):

```
brew install --cask docker
```

Termina en 2-3 minutos. Si no tienes Homebrew, ve a la opción siguiente.

Opción manual

Abre tu navegador y entra en:

```
https://www.docker.com/products/docker-desktop/
```

Pulsa:

- **Download for Mac - Apple Silicon** si tu Mac es M1/M2/M3/M4.
- **Download for Mac - Intel** si tu Mac es Intel.

Se descarga un fichero `Docker.dmg` (~500-700 MB). Doble click sobre él. Arrastra el icono de Docker a la carpeta Aplicaciones.

B.3 Primer arranque

Abre Aplicaciones → Docker. La primera vez:

- macOS pide confirmar que quieres abrir una aplicación descargada de internet. Confirma.

- Docker pide permiso para instalar componentes con privilegios de administrador. Introduce tu contraseña de usuario macOS.
- Aparece un acuerdo de licencia. Acéptalo.
- El icono de la **ballena** aparece arriba en la barra de menú (junto al reloj y el wifi). Cuando deja de animarse, Docker está listo.

B.4 Verificar

Abre Terminal (Cmd + Espacio, escribe “Terminal”, Enter):

```
docker --version
docker compose version
docker run --rm hello-world
```

Si los tres comandos responden con texto coherente, Docker funciona.

C • Linux

En Linux instalas el motor de Docker directamente, sin Desktop. Los comandos cambian según tu distribución.

C.1 Ubuntu / Debian

Abre una terminal y ejecuta línea por línea (te pedirá tu contraseña cuando uses `sudo`):

```
sudo apt update
sudo apt install -y docker.io docker-compose-plugin
sudo systemctl enable --now docker
```

El primer comando actualiza la lista de paquetes disponibles. El segundo instala Docker y el plugin de Compose. El tercero arranca el servicio de Docker y lo deja activado para arrancar automáticamente en cada inicio del sistema.

A continuación, añade tu usuario al grupo `docker` para no tener que escribir `sudo` cada vez:

```
sudo usermod -aG docker $USER
```

Para que el cambio de grupo tenga efecto, **cierra la terminal y abre otra nueva**. O alternatively:

```
newgrp docker
```

C.2 Fedora

```
sudo dnf install -y docker docker-compose-plugin
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
```

Cierra y abre la terminal.

C.3 Arch Linux

```
sudo pacman -S --noconfirm docker docker-compose
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
```

Cierra y abre la terminal.

C.4 Verificar

```
docker --version  
docker compose version  
docker run --rm hello-world
```

Los tres deben responder. Si el último imprime “Hello from Docker!”, estás listo.

D • Comprobación final (todos los sistemas)

Independientemente del sistema operativo, abre una terminal donde puedas ejecutar `docker` (en Windows con WSL: Ubuntu; en Windows nativo: PowerShell; en macOS y Linux: la terminal del sistema). Lanza estos cuatro comandos uno tras otro:

```
docker --version
docker compose version
docker info
docker run --rm hello-world
```

Lo que deberías ver:

- `docker --version` : una línea con la versión, por ejemplo `Docker version 25.0.3, build 4deb41`.
- `docker compose version` : una línea con la versión de Compose, por ejemplo `Docker Compose version v2.24.5`.
- `docker info` : un bloque grande con información del motor. Lo importante es que **no haya errores en rojo**. Las primeras líneas deben hablar de containers, imágenes, almacenamiento.
- `docker run --rm hello-world` : descarga una imagen pequeña y la ejecuta. Termina con “Hello from Docker!”.

Si los cuatro funcionan, tu sistema está perfecto para el curso.

E · Descargar y descomprimir el material del curso

Pide a tu profesor el fichero `Anbam-curso.zip`. Lo recibirás por correo, USB o enlace de descarga.

Dónde descomprimirlo

La ruta correcta depende del sistema operativo. Es **importante** elegirla bien porque Docker en algunos casos es muy lento si pones el material en la carpeta equivocada.

Sistema	Ruta recomendada	Por qué
macOS	<code>~/Anbam-curso</code>	Cualquier ubicación del home funciona bien.
Linux	<code>~/Anbam-curso</code>	Igual que macOS.
Windows + WSL	<code>~/Anbam-curso</code> dentro de Ubuntu	NO en <code>/mnt/c/</code> . Docker es muy lento si los volúmenes están en el disco de Windows.
Windows nativo	<code>C:\Anbam-curso</code>	Carpeta corta, sin acentos ni espacios.

Cómo descomprimir, paso a paso

En macOS

1. Localiza el `Anbam-curso.zip` en Finder (suele estar en Descargas).
2. Doble click sobre él. macOS lo descomprime automáticamente y crea una carpeta `Anbam-curso` al lado.
3. Arrástrala a tu home (`/Users/TuNombre/`).
4. Abre Terminal y verifica:

```
cd ~/Anbam-curso
ls
```

Debes ver una lista con `EMPIEZA_AQUI.md`, `setup.sh`, `labs/`, `material_alumno/`, etc.

En Linux

```
cd ~
unzip ~/Downloads/Anbam-curso.zip
cd Anbam-curso
ls
```

Misma salida que en macOS.

En Windows con WSL (recomendado)

El ZIP probablemente lo tienes en Descargas de Windows. Para llevarlo al sistema de archivos de Linux y descomprimirlo allí:

```
cd ~
cp /mnt/c/Users/$USER/Downloads/Anbam-curso.zip .
unzip Anbam-curso.zip
cd Anbam-curso
ls
```

Si `unzip` no está instalado en Ubuntu:

```
sudo apt install -y unzip
```

Y vuelve a ejecutar el `unzip` anterior.

En Windows nativo (sin WSL)

1. Localiza `Anbam-curso.zip` en el Explorador de archivos. Suele estar en `C:\Users\TuNombre\Downloads\`.
2. Click derecho sobre el ZIP → **Extraer todo**.
3. En la ventana, en “Los archivos se extraerán a esta carpeta”, borra la ruta sugerida y escribe `C:\Anbam-curso`.
4. Pulsa **Extraer**. Windows crea `C:\Anbam-curso\Anbam-curso\...` por defecto (carpeta dentro de carpeta). Mueve el contenido de esa subcarpeta a `C:\Anbam-curso\` para evitar la duplicación.
5. Abre PowerShell:

```
cd C:\Anbam-curso
ls
```

Debes ver `EMPIEZA_AQUI.md`, `setup.ps1`, `labs/`, etc.

F • Levantar el laboratorio del curso

Desde la carpeta donde descomprimiste el material, ejecuta un único comando:

macOS, Linux y Windows con WSL

```
./setup.sh
```

Windows nativo (PowerShell)

```
.\setup.ps1
```

Si Windows te dice que la ejecución de scripts está deshabilitada, abre PowerShell **como administrador** y ejecuta:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Confirma con **S** (sí). Cierra y vuelve a abrir PowerShell normal, y relanza `.\setup.ps1`.

Qué hace `./setup.sh` (o `setup.ps1`)

El script hace cinco cosas, en este orden:

1. **Comprueba que Docker está abierto y funciona.** Si no, te avisa y para. Si te paras aquí: asegúrate de que la ballena de Docker está fija en la bandeja.
2. **Descarga las cinco imágenes del laboratorio**, con reintentos automáticos si una falla:
 - `postgres:16-alpine` — la base de datos para MLflow.
 - `minio/minio` — almacenamiento tipo S3 para el curso.
 - `minio/mc` — cliente de MinIO, crea los buckets iniciales.
 - `ghcr.io/mlflow/mlflow` — el tracking server.
 - `jupyter/scipy-notebook` — entorno Jupyter con librerías científicas.

La primera vez tarda **5-10 minutos** y descarga unos 2 GB. La segunda vez, segundos.
3. **Arranca los contenedores con Docker Compose.** Define la red interna, los volúmenes persistentes y las dependencias entre servicios.
4. **Espera a que cada servicio responda correctamente.** Hace comprobaciones HTTP a cada URL y solo continúa cuando todos están sanos.
5. **Te muestra los enlaces** para abrir en el navegador y te indica el siguiente paso (lanzar el Lab 1).

Lo que debes ver al final

```
=====
LIST0. Abre estos enlaces en tu navegador:
=====

Jupyter   : http://localhost:8888   (token: anban)
MLflow    : http://localhost:5050
MinIO UI  : http://localhost:9001   (minio / minio12345)

Para hacer el Lab 1, ejecuta:
    cd labs/lab1_dataops && ./run.sh
=====
```

Comprobar que los tres servicios están vivos

Abre el navegador y entra a cada URL:

URL	Qué debes ver
http://localhost:8888	Una página de Jupyter pidiendo un token. Mete <code>anban</code> y pulsa Sign in. Aparece el explorador de ficheros de Jupyter.
http://localhost:5050	La interfaz de MLflow. Sin experimentos aún (los crearemos en el Lab 2).
http://localhost:9001	Una ventana de login de MinIO. Usuario <code>minio</code> , contraseña <code>minio12345</code> . Tras entrar, ves los buckets <code>datasets</code> y <code>mlflow</code> .

Si los tres responden, estás dentro del curso. **Felicidades:** la parte difícil ha terminado.

G · Si algo falla: el doctor automático

El curso incluye un script de diagnóstico que detecta y explica los problemas más comunes. Si en cualquier momento algo no funciona:

macOS, Linux y Windows con WSL

```
./doctor.sh
```

Windows nativo

```
.\doctor.ps1
```

Salida típica con todo bien:

```

=====
Diagnostico del laboratorio ANBAN
=====

- Sistema -
  · Sistema operativo: Linux 6.5.0
- Docker instalado -
  ✓ docker encontrado: Docker version 25.0.3
- Motor de Docker activo -
  ✓ el motor responde
- Docker Compose v2 -
  ✓ docker compose disponible
- Acceso a registries (Docker Hub, GHCR) -
  ✓ Docker Hub accesible
  ✓ GHCR (GitHub Container Registry) accesible
- Puertos del laboratorio -
  ✓ puerto 5050 ocupado por un contenedor del curso
  ✓ puerto 5432 ocupado por un contenedor del curso
  ✓ puerto 8888 ocupado por un contenedor del curso
  ✓ puerto 9000 ocupado por un contenedor del curso
  ✓ puerto 9001 ocupado por un contenedor del curso
- Contenedores del curso -
  ✓ anban-postgres arriba
  ✓ anban-minio arriba
  ✓ anban-mlflow arriba
  ✓ anban-jupyter arriba
  ✓ anban-minio-init termino correctamente
- Servicios HTTP del laboratorio -
  ✓ Jupyter responde
  ✓ MLflow responde
  ✓ MinIO API responde
  ✓ MinIO UI responde
- Espacio en disco -
  ✓ 50G disponibles

=====
Todo en orden. No hay problemas detectados.
=====

```

Si hay algún problema, el doctor te lo dice claramente con una marca **X** y una línea “arreglo:” que te explica qué hacer.

H · Problemas frecuentes y cómo solucionarlos

Docker no arranca o la ballena queda animada eternamente

Causa probable 1: virtualización deshabilitada en BIOS

Es la causa más habitual en Windows. Ve a la sección “Activar virtualización en la BIOS” del capítulo A.

Causa probable 2: Hyper-V deshabilitado por antivirus

Algunos antivirus (Avast, McAfee) o programas de virtualización viejos (VirtualBox antiguo) bloquean Hyper-V. Pasos:

1. Desinstala VirtualBox/VMware si los tienes y no los necesitas.
2. Desactiva temporalmente el antivirus.
3. Reinicia.
4. Vuelve a arrancar Docker Desktop.

Causa probable 3: WSL desactualizado

Solo en Windows. Abre PowerShell como administrador:

```
wsl --update  
wsl --shutdown
```

Y reabre Docker Desktop.

“Cannot connect to the Docker daemon”

Significa que el motor de Docker no está arrancado.

- **macOS / Windows:** abre Docker Desktop manualmente desde el menú Inicio o Aplicaciones.
- **Linux:** arranca el servicio:

```
sudo systemctl start docker
```

Una imagen no se descarga (timeout, 500 Internal Server Error)

Tu red está bloqueando algún registry de Docker.

1. Cierra Docker Desktop (Quit Docker, esperar a que la ballena desaparezca).
2. Reabre Docker Desktop. Espera a que la ballena se fije.

3. Lanza `./setup.sh` (o `.\setup.ps1`) otra vez.

Si sigue fallando: tu red corporativa o de hotel bloquea `*.docker.io` o `*.ghcr.io`. Cambia de red (red de datos del móvil compartida, por ejemplo). Si funciona ahí, ya sabes que era la red. Pide a IT que abra los registries en la red de la oficina.

Puerto ya ocupado por otro proceso

Si al ejecutar `./setup.sh` te aparece un error tipo:

```
Bind for 0.0.0.0:5050 failed: port is already allocated
```

Significa que algún otro programa está usando ese puerto.

Para averiguar cuál:

- **Linux / macOS:**

```
lsof -i :5050
```

- **Windows PowerShell:**

```
Get-NetTCPConnection -LocalPort 5050
```

Cierra la aplicación que esté usando ese puerto y vuelve a lanzar `./setup.sh`. Si necesitas mantener ambos programas, cambia el puerto en `docker/docker-compose.yml` (líneas `ports:`).

El laboratorio funciona pero un alumno tiene una pantalla negra al abrir Jupyter

Tu navegador necesita el token. Cuando abres `http://localhost:8888` te pide un token: introduce `anban` (en minúsculas) y pulsa “Log in”.

Ejercicio para fijar lo aprendido

Para asegurarte de que todo está correcto, vamos a hacer un pequeño ejercicio. En tu terminal, ejecuta:

```
docker ps
```

Tienes que ver **cuatro contenedores** del curso en estado “Up”. Si ves:

NAME	STATUS	PORTS
anban-jupyter	Up 2 minutes	0.0.0.0:8888->8888/tcp
anban-mlflow	Up 2 minutes	0.0.0.0:5050->5000/tcp
anban-minio	Up 2 minutes	0.0.0.0:9000-9001->9000-9001/tcp
anban-postgres	Up 2 minutes	0.0.0.0:5432->5432/tcp

todo está correcto. Anota la columna `STATUS` de cada uno. Ese es el estado normal del laboratorio. Cuando los nombres aparezcan en `Exited`, sabrás que algo se cayó y tendrás que volver a hacer `./setup.sh`.